Système d'exploitation : TP0
janvier 2026 — Pierre Rousselin

1 Arguments de la ligne de commande et environnement d'un processus

Mots-clés :

— arguments de la ligne de commande
— environnement
— arithmétique des pointeurs
— double-pointeurs

Exercice 1 : `mon_echo`

1. Soit la ligne de commande

```
$ /bin/echo 12 lol 'a c'
```

Que valent `argc` et `argv` dans ce cas ? Représenter le tableau `argv` et les chaînes pointées.

2. Écrire votre propre version de `echo` : un programme qui affiche ses arguments, chaque argument étant séparé du suivant par un espace, avec un saut de ligne à la fin.

3. Même question, en utilisant uniquement `argv` et en utilisant uniquement `putchar` pour les sorties.

--- * ---

Exercice 2 : Environnement

L'environnement d'un processus est un ensemble de chaînes de caractères du type `CLE=valeur` (par exemple `HOME=/home/alice`).

L'environnement d'un processus peut influer sur le comportement de ce processus (par exemple en changeant la langue des messages écrits pour l'utilisateur, ...) et a ceci de particulier qu'il est, par défaut, transmis aux processus enfants.

L'environnement d'un processus est accessible en C en déclarant la variable globale

```
extern char **environ;
```

La variable `environ` est un pointeur vers un tableau de pointeurs vers des `char` (des chaînes de caractères terminées par des caractères nuls) dont le dernier élément est `NULL`

— Faire un dessin, dans le cas où l'environnement ne contient que `PATH=/usr/bin` et `HOME=/home/alice`.
— Écrire un programme `environ.c` qui affiche son environnement comme le fait la commande `env` lorsqu'elle est lancée sans argument.
— Tester avec

```
$ LOL=rofl ./environ
$ env -i ./environ
$ env -i PLOP=hop CC=c99 ./environ
$ env -u HOME -u USERNAME ./environ
$ export A=coucou
$ ./environ
```

Voir la page de manuel de `env` pour comprendre les options `-i` et `-u`. Voir `help export` si vous ne connaissez pas `export`.

--- * ---

2 Lire un répertoire, appels système `opendir`, `readdir` et `closedir`

Mots-clés :

— structures

— pointeurs

— fichiers et répertoires sous Unix

— autonomie et manuel

Exercice 3 : `mon_ls`

Le but du jeu est d'écrire une version minimale du programme `ls` qui liste les fichiers d'un ou plusieurs répertoires.

Dans le standard Unix, on interagit avec un répertoire via un pointeur vers un type opaque `DIR`. Il faut ouvrir le répertoire d'abord. Ici par exemple, on ouvre le répertoire racine.

```
DIR *d = opendir("/");
```

Ensuite, les différentes entrées du répertoire seront des pointeurs vers un type transparent `struct dirent`, on les obtient par des appels successifs à la fonction `readdir`.

```
struct dirent *ent = readdir(d);
```

Les seuls champs de cette structure qui sont spécifiés par POSIX sont `d_name` (le nom de base du fichier) et `d_ino` (le numéro d'inode).

Lorsqu'on ne l'utilise plus, le répertoire doit être fermé :

```
closedir(d);
```

Pour les pages de manuel concernées (que vous aurez sans doute à lire) :

```
$ man opendir
$ man 3 readdir
```

1. Quels fichiers d'en-tête faut-il `#include` ? (La réponse est dans le manuel).
2. Écrire une première version de `mon_ls` qui prend un argument : le nom du répertoire à lister et affiche les noms des fichiers contenus dans ce répertoire, à raison d'un par ligne. Bien sûr, tester votre programme, comparer avec la sortie de `ls`.
3. Apporter quelques améliorations :
   a) utiliser la valeur de retour de `opendir` pour détecter une erreur, afficher un message d'erreur pertinent avec `perror` (voir `man perror`). Tester avec un répertoire qui n'existe pas et un répertoire sur lequel vous n'avez pas la permission `r`.
   b) Permettre à `mon_ls` d'avoir 0 argument. Dans ce cas, lister les fichiers du répertoire courant.
   c) Permettre à `mon_ls` d'avoir plus d'un argument. Dans ce cas, les noms des répertoires apparaissent sur une ligne, seuls, avant les fichiers qu'ils contiennent, et les contenus des différents répertoires sont séparés par une ligne vide. Voir `ls / /usr | cat` pour un exemple.
   d) Ne pas afficher les noms des fichiers cachés.
   e) Éventuellement, ajouter les options `-a` pour afficher les noms des fichiers cachés, et `-i` pour les numéros d'inode.

--- * ---

3 Programmer `rev` avec `alloc` et `realloc`

Mots-clés :

— lecture et écriture caractère par caractère

— allocation dynamique

Exercice 4 : `mon_rev`

Le programme `rev`, non standard mais très répandu affiche le contenu de fichiers (ou bien de son entrée standard) ligne par ligne en inversant l'ordre des caractères de chaque ligne. Voici un exemple où les lignes 1, 3 et 5 sont celles entrées par l'utilisateur, les autres sorties par `rev`. À la fin, l'utilisateur a tapé Ctrl-D pour signaler une fin de fichier.

```
$ rev
salut !
! tulas
lol
lol
malloc et realloc
collaer te collam
```

Vous allez programmer votre propre version de `rev` avec les contraintes et conseils suivants :

— Pour lire un caractère d'entrée, on utilisera la fonction `getchar` qui s'utilise de la façon suivante :

```c
#include <stdio.h>
int main(void)
{
        int c;
        c = getchar();
        if (c != EOF)
                putchar(c);
        return 0;
}
```

Le programme précédent lit un seul caractère de l'entrée standard. Si l'on est déjà à la fin du fichier (c'est-à-dire, ici, dans le cas d'un fichier vide) c'est la constante EOF qui est retournée par `getchar`. Dans le cas contraire le caractère est imprimé.

Remarque technico-technique désagréable : la variable qui reçoit la valeur de retour de `getchar` *doit* être de type `int`, car il n'est pas garanti que le type `char` puisse recevoir `EOF` (dans le cas où `char` serait non signé).

— Chaque caractère d'une ligne est stocké dans un tableau *sur le tas*. Ce tableau a initialement une taille de `80`, mais si cela ne suffit pas, vous devrez multiplier sa taille par 2, à l'aide de `realloc`, chaque fois que nécessaire. N'oubliez pas de libérer ce tableau à la fin du programme.

— Lorsque le flux atteint la fin d'une ligne ou la fin du fichier, il faut afficher à l'envers le contenu du tableau.

— Le même tableau sera utilisé pour toutes les lignes.

— À la fin du programme, pour des raisons pédagogiques, afficher la taille finale du tableau utilisé.

Quelques tests à faire pour vérifier la robustesse du programme :

```
$ gcc -Wall mon_rev.c -o mon_rev
$ ./mon_rev <mon_rev.c
$ ./mon_rev </usr/include/stdio.h
$ tr -d '\n' </usr/include/stdio.h | ./mon_rev
$ cat /usr/include/*.h | tr -d '\n' | ./mon_rev
$ gcc -Wall -g mon_rev.c -o mon_rev
$ tr -d '\n' </usr/include/stdio.h >tmp; valgrind ./mon_rev tmp
```

La dernière commande ne doit pas détecter de fuite de mémoire (*memory leak*) en dehors de celles de `tr`.

`--- * ---`